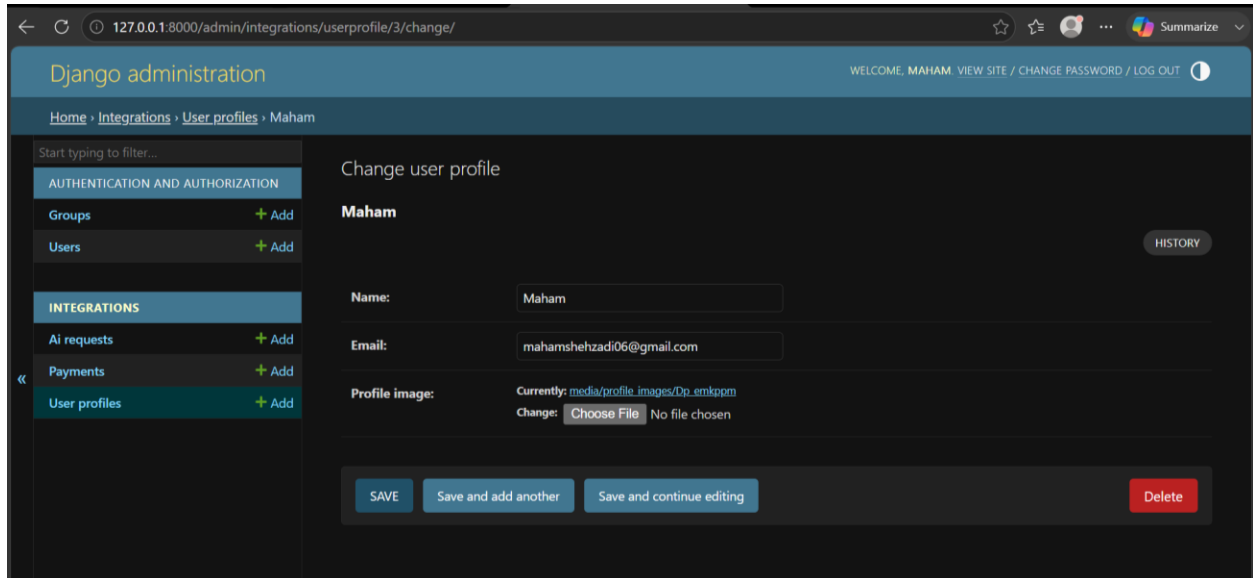
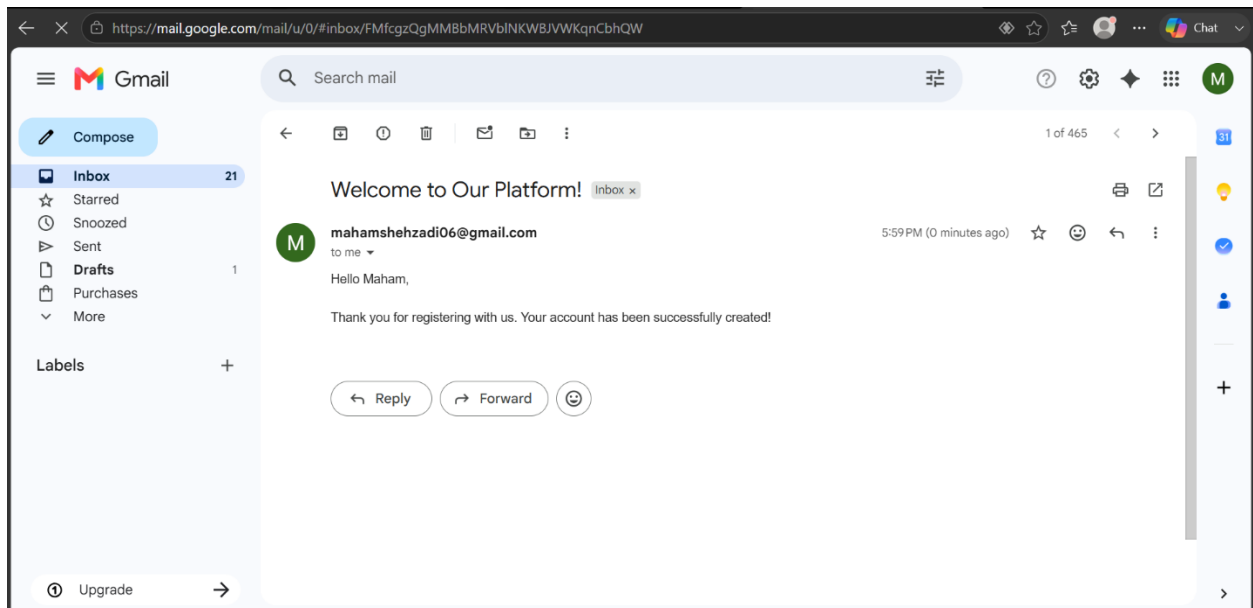


Django Backend Integration Project

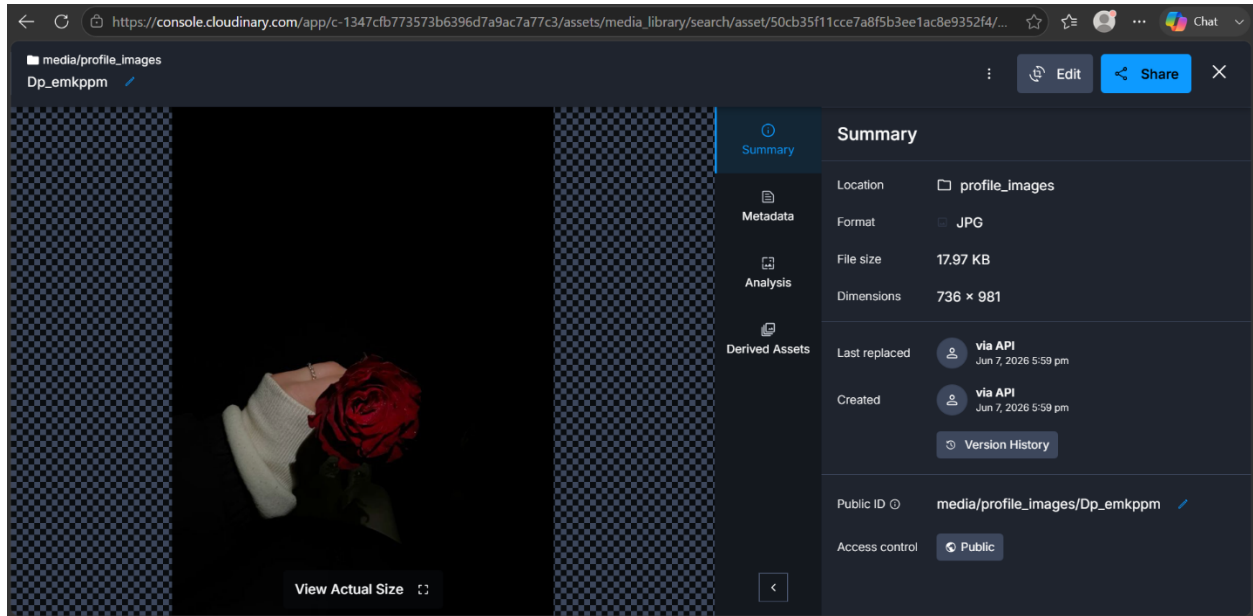
Adding user:



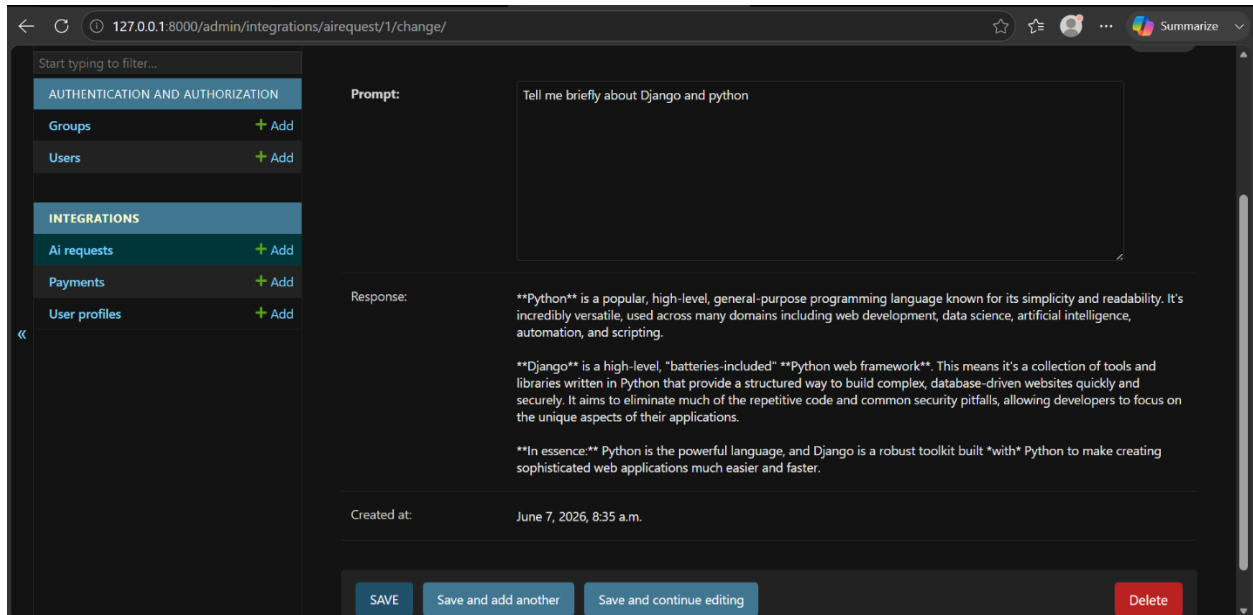
Welcome message on user registration:



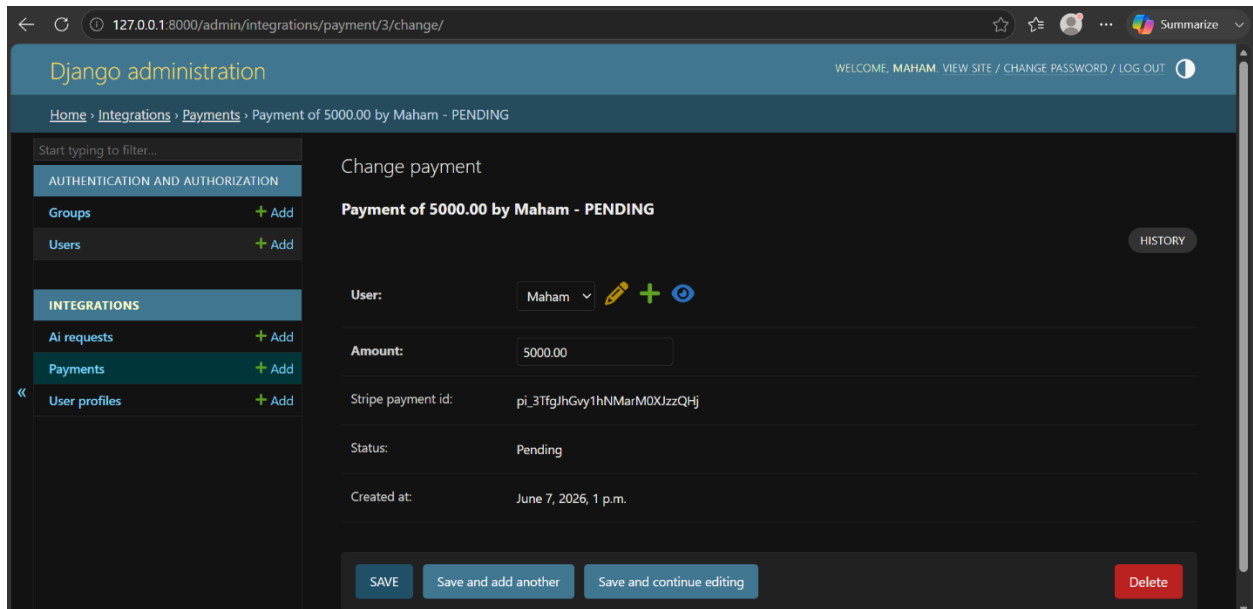
Upload profile image on Cloudinary:



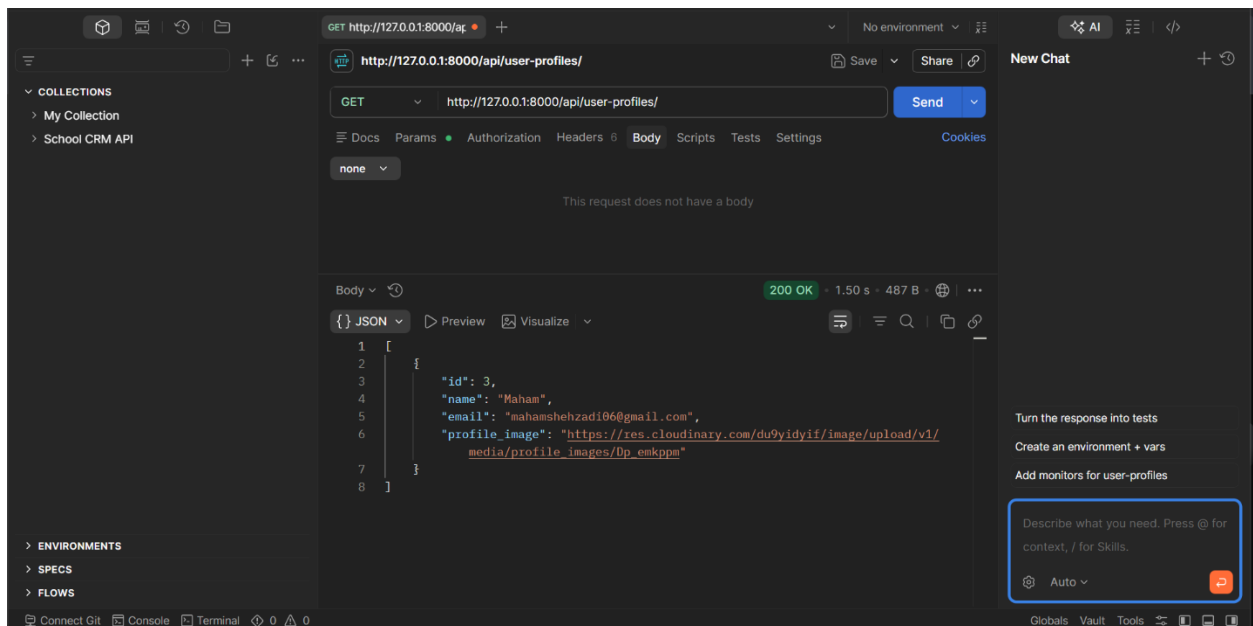
AI Request Response through Gemini:

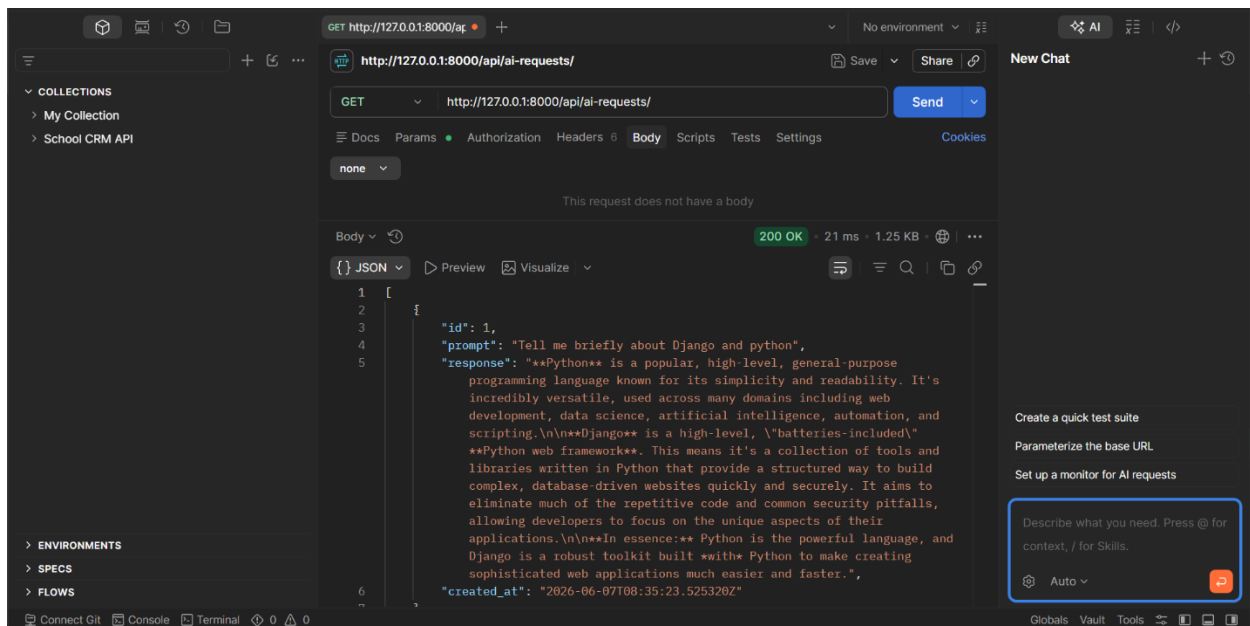
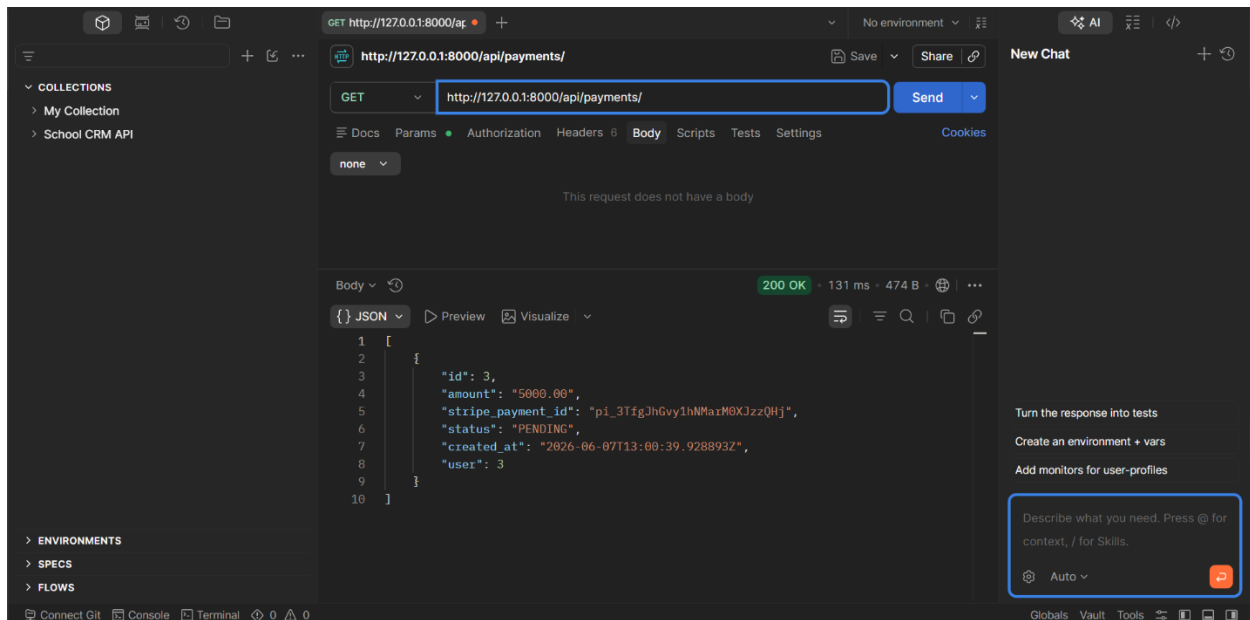


Payment module:



Postman Testing:





Errors and Solutions during Task:

1. Typo in DecimalField Configuration (Field Error):

- **Error:** `TypeError: Field.__init__() got an unexpected keyword argument 'decimal_digits'`
- **Reason:** In the Payment model inside `integrations/models.py`, the parameter for specifying decimal precision in Django's `DecimalField` was mistakenly written as `decimal_digits`. Django does not recognize this keyword argument.

- **Resolution:** The file was opened, and `decimal_digits=2` was replaced with Django's standard argument `decimal_places=2`. After making this correction, the `makemigrations` command executed successfully.

2. Missing Pillow Library (System Check Error)

- **Error:** `SystemCheckError: ... Cannot use ImageField because Pillow is not installed.`
- **Reason:** Django requires an external Python imaging library called Pillow to handle image processing, resizing, and validation when using `ImageField` in models (such as the `UserProfile` model). The library was missing from the virtual environment.
- **Resolution:** The terminal was used to install the required library inside the virtual environment (venv) by running the command `pip install Pillow`. This successfully activated Django's image handling system.

3. Google Gemini SDK Deprecation & Attribute Error

- **Error:** `AttributeError: module 'google.genai' has no attribute 'configure' (along with a FutureWarning about deprecation).`
- **Reason:** The older package `google.generativeai` has been deprecated by Google. After upgrading to the modern package `google-genai`, the code in `integrations/signals.py` was still trying to use the legacy setup method `genai.configure(api_key=...)`. The new SDK does not contain a `configure` attribute and uses a centralized client instance instead.
- **Resolution:** The line containing `genai.configure` was entirely removed from `signals.py`. The asynchronous webhook listener (`call_gemini_api`) was updated to initialize the official client structure using `client = genai.Client(api_key=os.getenv('GEMINI_API_KEY'))` and updated to call the standard `gemini-2.5-flash` model.

4. Cloudinary Missing API Secret & Invalid Signature (Storage Authorization Error)

- **Error:** `ValueError: Must supply api_secret followed by AuthorizationRequired: Invalid Signature.`
- **Reason:** Initially, Django couldn't read the Cloudinary credentials due to the malformed syntax in `settings.py`. After setting up the `.env` file, the wrong API Secret was copied from the Cloudinary dashboard (a sub-feature key from the `mediaflows` or `moderation` row instead of the actual master environment credentials), which caused Cloudinary to reject the upload signature.

- **Resolution:** The Cloudinary console was accessed, and the actual **Root** API Secret was revealed and copied from the Product Environment Credentials section. It was correctly saved in the .env file under CLOUDINARY_API_SECRET. After restarting the Django development server, the application successfully authorized connection, and user images were uploaded straight to the online Cloudinary storage without errors.

5. Missing Imports in Project Configuration (NameError)

- **Error:** NameError: name 'UserProfileListAPIView' is not defined
- **Reason:** When setting up new endpoints for Postman tracking inside core/urls.py, the Django generic views were added directly to the urlpatterns list, but the Python script did not import them at the top of the file. Because of this, the URL router could not identify where the view functions were located.
- **Resolution:** Clean import statements were explicitly declared at the top of core/urls.py: from integrations.views import UserProfileListAPIView, PaymentListAPIView, AiRequestListAPIView. This resolved the execution dependencies.

6. Explicit Model Name Casing Conflict (ImportError)

- **Error:** ImportError: cannot import name 'AiRequest' from 'integrations.models'
- **Reason:** The REST API views and serializers tried to explicitly import AiRequest. However, due to character casing differences in the actual integrations/models.py file (e.g., spelled as AIRequest or AI_Request), the strict module compiler failed to resolve the name and crashed the runserver instance.
- **Resolution:** To make the architecture robust and bypass precise casing errors, Python's dynamic model lookup was implemented inside views.py and serializers.py using Django's built-in applications registration: apps.get_model('integrations', 'UserProfile'). An automated pattern-matching lookup script was added to find the AI log model automatically regardless of camelCase variations. This completely bypassed the string conflict and successfully booted the development server.